

Smart Contract Assessment

Moonwell

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

SOURCE CODE
mamo-contracts

DATE OF ENGAGEMENT
Aug 25, 2026 - Aug 31, 2026

ASSESSED COMMIT ID
5a29c36

SEVERITY BREAKDOWN



INTRODUCTION

Moonwell engaged Halborn to perform a security assessment of their smart contracts starting on August 26th, 2026 and ending on August 31st, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The Moonwell (Mamo) codebase in scope consists of smart contracts implementing a leveraged Aerodrome Slipstream concentrated-liquidity yield strategy over Moonwell lending, featuring a per-user vault/strategy clone model with owner-staged venue migration, per-cycle asymmetric range placement, idle-USDC parking, request-based redemptions with frozen recipients, and a permissionless deleverage safety valve pinned below Moonwell's collateral factor.

ASSESSMENT SUMMARY

Halborn was allocated 4 days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in

blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which were mostly addressed by the **Moonwell** team. The main recommendations were:

- **Fail-closed on the dust regime before the caller-floor guard, and fail-soft on the reward-feed reads so an unrelated feed's heartbeat cannot gate the migration path.**
- **Compare against the pre-crystallize supply, matching the executed path's pre-crystallize capture.**
- **On the sole-holder full-redeem path, pay the phantom-fee delta to the redeemer or to the fee recipient rather than let it strand.**

I TEST APPROACH AND METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (**Foundry**).
- Fork testing against main networks (**Foundry**).

- Static security analysis of scoped contracts, and imported functions (Slither).

4 Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = \max(m_I) + \frac{\sum m_I - \max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
----------	------	--------	-----	---------------

9 - 10

7 - 8.9

4.5 - 6.9

2 - 4.4

0 - 1.9

Scope

REPOSITORY



(a) Repository:  **mamo-contracts**

(b) Assessed Commit ID:  5a29c366d2b688a6209b9418797e39a05b1effce

(c) Items in scope:

✓  src 6 files

✓  leveraged-aero 4 files

 <> LeveragedAerodromeCLStrategy.sol Solidity

 <> LeveragedAeroFees.sol Solidity

 <> LeveragedAeroManager.sol Solidity

 <> LeveragedAeroVenue.sol Solidity

 <> LeveragedAeroVault.sol Solidity

 <> MamoLeveragedAeroStrategy.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:



 3d0a973

 756b998

 414f04d

 9d4cab4

 cc0dd31

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Reward-token donation breaks flatten's idempotency and DoS-es migrateVenue	● Low	2.5	RISK ACCEPTED 09/08/2026
HAL-002	previewRedeem disagrees with redeem on the full-redeem flag when crystallize defers	● Low	2.5	RISK ACCEPTED 09/08/2026
HAL-003	Sole-holder full-redeem with deferred crystallize strands the phantom-fee delta as un-attributable idle USDC	● Low	2.5	RISK ACCEPTED 09/08/2026
HAL-004	setMaxLtv is permanently locked after a Moonwell CF cut	● Low	2.5	SOLVED 09/04/2026
HAL-005	migrateVenue is blocked by the same CF-cut gate, invalidating staged multisig migrations	● Low	2.5	SOLVED 09/04/2026
HAL-006	Stale stagedVenueHash survives an Ownable2Step vault-owner rotation, letting the proposer fire a venue rewrite the new owner never authorised	● Low	2.0	SOLVED 09/04/2026
HAL-007	withdrawIdle fallback loosens the LTV bound when Moonwell prices USDC above \$1	● Informational	1.7	SOLVED 09/04/2026
HAL-008	applyVenue silently rewrites minHealthBps on migration without a dedicated event	● Informational	1.7	SOLVED 09/04/2026
HAL-009	_settleShortfall reads every leg's Chainlink feed, so a stale sibling-leg feed reverts emergencyRedeem (deadman), flatten, and	● Informational	1.7	SOLVED 09/04/2026

#	TITLE	SEVERITY	SCORE	STATUS
	migrateVenue on a zero-debt leg			
HAL-010	Redemption recipient is snapshotted at request time, letting an outgoing account owner capture funds meant for the new owner	Informational	1.7	ACKNOWLEDGED 09/08/2026
HAL-011	Proposer can DoS an owner-staged cross-tickSpacing migration by poisoning \$.skewBps between stage and migrate	Informational	1.1	ACKNOWLEDGED 09/08/2026

Findings & Tech Details

HAL-001

RISK ACCEPTED

Reward-token donation breaks flatten's idempotency and DoS-es migrateVenue

Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`LeveragedAeroVenue._sellRewardBalance` is invoked from `flattenImpl` with `callerFloorRequired = true` and is not wrapped in a `try/catch`. The function short-circuits only in two places:

- An oracle-free dust band: `if (bal < 1e20 / REWARD_PRICE_CEILING_USD8) return;` (line 507, i.e. `bal < 1e8`).
- A priced-floor check: `if (floor == 0) return;` (line 519) — which runs after two `readUsd8` calls on `$.aeroUsdFeed` and `$.usdcFeed`.

The `ZeroMinOut()` guard sits after the `floor == 0` short-circuit:
solidity

SOL 502-509

```
502 if (bal < 1e20 / REWARD_PRICE_CEILING_USD8) return;
503 uint256 pUsdc8 = LeveragedAeroValuation.readUsd8($.usdcFeed, ...);
504 uint256 floor = Math.mulDiv(Math.mulDiv(bal, readUsd8($.aeroUsdFeed, ...), 1e18), 1e6,
    pUsdc8)
505     * (10000 - $.maxSlippageBps) / 10000;
506 if (floor == 0) return;
507 if (callerFloorRequired && minRewardUsdcOut == 0) revert ZeroMinOut();
508 uint256 usdcOut = LeveragedAeroValuation.swapAeroToUsdc(rewardTok, $.usdc, bal,
    minRewardUsdcOut);
509 if (usdcOut < floor) revert BelowOracleFloor();
```

`flatten` is proposer-callable and the only path to a flat book, and `migrateVenue` enforces `BookNotFlat` at `migrateImpl`, so `migrateVenue` cannot execute until `flatten` succeeds. The docstring at lines 493–494 explicitly claims that a rounded-to-zero floor keeps `flatten` idempotent so "a 1e6-wei donation [does not] brick `flatten`," but the guard only holds inside the untyped dust band; a slightly larger reward-token donation moves the balance into the priced regime where three distinct externally-triggerable conditions each turn `flatten` into a revert.

The attacker is any EOA: the gauge's reward token is a standard ERC-20 whose `balanceOf(venue)` this function reads, so anyone can send AERO (or the pool's reward token) directly to the strategy contract. There is no cost gate beyond the ERC-20 transfer itself. Concretely, with AERO $\approx$ \$1 and `maxSlippageBps  $\leq$  1000`, `bal  $\approx$  1e14` wei ($\approx$ 1e-4 AERO, sub-cent value) both passes the dust band ($\geq$ 1e8) and yields a non-zero floor. Once the priced regime is entered, the following DoS surfaces open:

Scenario 1 - realistic, single-tx bricking of `flatten(0, minIdleUsdcOut)`

The proposer calls `flatten(minRewardUsdcOut = 0, minIdleUsdcOut)`, which is the natural call for a book the proposer believes is already flat (the docstring at line 454 advertises idempotence on a flat book). Steps:

1. Attacker transfers $\approx$ 1e14 wei of the gauge reward token to the venue address, a permissionless ERC-20 transfer, cost $\approx$ one gas + dust.
2. Proposer calls `flatten(0, minIdleUsdcOut)`.
3. `_sellRewardBalance(0, true)` is entered; `bal = 1e14  $\geq$  1e8` clears line 507; both `readUsd8` calls succeed; `floor > 0` so line 519 does not return.
4. Line 520 fires `revert ZeroMinOut()`.

The book is stuck in the same state the proposer expected to be a no-op, and every retry with `minRewardUsdcOut = 0` reverts identically until the proposer changes their call signature. `migrateVenue` inherits the block because `migrateImpl`'s `BookNotFlat` requires the `flatten` to have completed on a live book.

Scenario 2 - feed staleness weaponised by the same donation

Once `bal  $\geq$  1e8`, execution is forced through both `readUsd8` calls at lines 511 and 514 before any exit is possible. If either `$.aeroUsdFeed` or `$.usdcFeed` is stale beyond `$.maxDelay` (or the sequencer is down / inside `gracePeriod`), `ChainlinkReader.readUsd` reverts with `StaleOracle` / `SequencerDown` / `GracePeriodNotOver` (`src/leveraged-aero/sherwood/libraries/ChainlinkReader.sol:20-39`). Without the donation, the dust-band exit at line 507 short-circuits before touching any feed; with the donation, `flatten` and hence `migrateVenue` gate on the liveness of both reward-side feeds, feeds unrelated to the position being unwound, for the whole staleness window. Chainlink's AERO/USD feed carries a heartbeat that is materially longer than the CL leg feeds this venue actually depends on for valuation, so the donation converts an unrelated feed's heartbeat into a live DoS on the migration path.

Scenario 3 - Aerodrome v2 push into `BelowOracleFloor`

With `bal` above the dust band and a non-zero floor, `swapAeroToUsdc` routes the whole balance through the AERO-v2 pool (`AERO_V2_ROUTER` / `AERO_V2_FACTORY`) with the caller's `minRewardUsdcOut` as the router bound. An attacker who front-runs `flatten` with a same-block

push against the AERO/USDC v2 pool can drive the realised fill below the post-checked oracle floor at line 522 (`if (usdcOut < floor) revert BelowOracleFloor();`), then unwinds the push in a back-run. The reward tranche the attacker donated is what forces `flatten` into the swap in the first place, so this is a self-triggered sandwich rather than a race against genuine harvest cash flow.

The net effect across all three scenarios is that a permissionless dust transfer converts the docstring-idempotent `flatten` into a repeatedly-revertable call, and `migrateVenue` (which cannot run without a flat book) inherits every revert path. The proposer is not funds-at-risk; the impact is loss of the "retry with the same args" operational property for the venue-migration workflow.

RECOMMENDATION

Fail-closed on the dust regime before the caller-floor guard, and fail-soft on the reward-feed reads so an unrelated feed's heartbeat cannot gate the migration path.

REMEDIATION COMMENT

RISK ACCEPTED: The `Moonwell` team made a business decision to risk accept this finding and not alter the contracts, stating that:

ZeroMinOut above the dust band is intentional and tested: a reward balance above the band is a real tranche and `flatten(1, ...)` clears it. `migrateVenue` has no reward-balance gate, so the migration DoS does not exist, and the oracle floor is checked post-swap, so a sandwich causes a revert, not a loss. Docstrings and runbook made precise in 1a49e5e

previewRedeem disagrees with redeem on the full-redeem flag when crystallize defers

• Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`LeveragedAeroVenue.previewRedeemImpl` is the advisory quote for the strategy's redeem entry point; by its own docstring (lines 1130-1139) it is intended to "mirror `redeem` EXACTLY, including the pending fee crystallise to the wei at the same `block.timestamp`, deferred-crystallise path included." The full-redeem flag is where the two diverge:

- `previewRedeemImpl` at line 375 computes the flag against `supplyPost` (the post-simulated-crystallise supply): `if (shares == supplyPost && _layout().tokenId != 0) return (assetsOut, false);`
- `LeveragedAerodromeCLStrategy.redeem` at line 1098 captures the flag against the pre-crystallize `totalSupply()`: `bool fullRedeem = shares == supply;` — where `supply` at line 1095 is read after the best-effort crystallise runs at line 1090, and is only overwritten with the simulated post-crystallise supply at line 1105 if the crystallise failed (the `!crystallized` branch of `_simulateCrystallize`).

Under the "crystallise succeeded" path the two supplies are the same (the mint has already happened, so live `totalSupply()` equals `supply + feeShares`, and the preview's simulation converges on the same figure). Under the "crystallise failed" path, H3 in the docs (vault issuance paused or fee recipient de-whitelisted, as documented at strategy line 781-786), the two diverge: preview compares against `supply + phantomFeeShares`, but execution compares against the raw `supply`. So for a sole shareholder (whose `shares == supply`) holding a book with a live LP (`tokenId != 0`) at the moment crystallise defers:

- **Preview:** `shares (== supply) != supply + phantomFeeShares` $\Rightarrow$ the guard does NOT fire, preview returns `(assetsOut, fastOk = true)`.
- **Execution:** `fullRedeem = (shares == supply)` is TRUE, so `fastRedeemImpl` is called with `isFullRedeem = true` and hits: `if (isFullRedeem && $.tokenId != 0) revert FastRedeemExceedsLtv(uint256.max, uint256($.maxLtvBps));`

Net: the preview advertises `fastOk = true`, `redeem(shares)` reverts

`FastRedeemExceedsLtv(uint256.max, maxLtvBps)`. The redeemer's UX contract, "if `fastOk ==`

`true`, the redeem will succeed at this block, else route to `requestRedeem` " is broken exactly in the crystallise-deferred branch it was written to cover.

RECOMMENDATION

Compare against the pre-crystallize supply, matching the executed path's pre-crystallize capture.

REMEDIATION COMMENT

RISK ACCEPTED: The `Moonwell` team made a business decision to risk accept this finding and not alter the contracts, stating that:

Moot at HEAD: the share-minting management/performance fee crystallize (`_crystallizeBestEffort`, `_simulateCrystallize`, `hwmPerShare`) was removed in b518af7 (PR #88) before this assessment landed. `previewRedeem` and `redeem` now read the same `totalSupply()` word and cannot diverge; the flag behaviour is pinned by existing tests.

Sole-holder full-redeem with deferred crystallize strands the phantom-fee delta as un-attributable idle USDC

• Low · 2.5 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

`LeveragedAerodromeCLStrategy.redeem` is the everyday oracle-priced exit. When `_crystallizeBestEffort` returns `false`, because vault issuance is paused (`strategyMint` is gated on the vault's `whenNotPaused`) or the fee recipient has been de-whitelisted, the fee mint is deferred but the redeem still proceeds. Then simulates the crystallise that could not run: `supply = _simulateCrystallize(navPre, supply);`, giving `supply' = S + phantomFeeShares`. Line 1107 prices the payout on the simulated book: `assetsOut = shares · navPre / (S + phantomFeeShares)`.

`fullRedeem`, however, was captured against the pre-crystallize `supply == S`, that capture is deliberate (per the same file's inline note, "a fact about the share ledger"), and correctly identifies the sole-shareholder full-redeem case even under a deferred crystallise. So for a sole holder with `shares == S` under deferred crystallise on a flat, zero-debt book (`tokenId == 0`, `debtUsdc == 0`, `phantomFeeShares > 0`):

- `fullRedeem = true`.
- `assetsOut = navPre · S / (S + phantomFeeShares) < navPre`.
- `LeveragedAeroVenue.fastRedeemImpl` enters its full-cToken-burn branch because `isFullRedeem && debtUsdc == 0 && $.tokenId == 0` holds. It burns the whole cToken balance and pays `payout = assetsOut + (realised - collateralUsdc)`, where `realised - collateralUsdc` is only the fresh-rate cToken surplus (interest accrued since `exchangeRateStored`), a distinct quantity from the phantom-fee delta.
- Burns all remaining shares: `strategyBurn(shares);` post-op `totalSupply() == 0`.
- The strategy retains `navPre - assetsOut = navPre · phantomFeeShares / (S + phantomFeeShares)` as idle USDC, the phantom-fee delta.

This residual is (a) not paid to the redeemer (they took `assetsOut`, which under-prices by exactly this factor); (b) not paid to the fee recipient, the deferred mint cannot happen after the exit because `_crystallizeFees` short-circuits when `supply == 0` (the "EMPTY BOOK" branch), zeroing `hwmPerShare` and advancing `lastFeeAccrualTimestamp`; the pending fee claim is erased. On the first re-entrant deposit while `totalSupply == 0`, the depositor mints against `nav = X + residual`

using the vault's virtual-offset first-deposit formula, effectively pulling the residual into their own share basis. The value is silently transferred to the first re-entrant depositor rather than to the fee recipient.

RECOMMENDATION

On the sole-holder full-redeem path, pay the phantom-fee delta to the redeemer or to the fee recipient rather than let it strand.

REMEDATION COMMENT

RISK ACCEPTED: The **Moonwell** team made a business decision to risk accept this finding and not alter the contracts, stating that:

Moot at HEAD: the fee crystallize layer was removed in b518af7 (PR #88); redeem prices shares on the live supply, so a sole-holder full redeem pays navPre exactly and nothing strands. The vault also has no pause, so the deferral precondition does not exist.

setMaxLtv is permanently locked after a Moonwell CF cut

• Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `setMaxLtvImpl` function re-derives the live Moonwell collateral factor on every call and passes it into `LeveragedAeroValuation.checkLtvBand`. That check's fifth rung reverts with `DeleverageTriggerAboveCF` whenever $\text{minHealthBps} \times \text{liveCF} \leq 1e8$, a condition that depends only on the stored `minHealthBps` and the current on-chain CF, never on the `maxLtvBps_` argument the admin is passing in. Once Moonwell governance cuts USDC's CF far enough to trip that product, every subsequent call to `setMaxLtv` reverts, including calls that lower the ceiling to a safer value. `setTargetLtv` skips this check but only changes policy, not the live position, and `adjustLeverage` is gated `onlyProposer`, not `onlyAdmin`, leaving the admin with no working lever to de-risk the vault.

The impact is a fully disabled risk-management control: once a CF cut trips the rung-5 gate, the venue admin has no on-chain path to lower `maxLtvBps` in response, even though lowering it is precisely the safe action the function exists to allow. The vault continues operating at the pre-cut ceiling indefinitely, and the only recovery is a destructive `flatten`-and-redeploy cycle, which carries its own operational and gas overhead and temporarily removes the venue from active management while a market-driven risk (a CF cut) is actively unresolved.

RECOMMENDATION

Make rung 5 evaluate the admin's intended target rather than the live CF-only product, e.g. only block increases to `maxLtvBps_`, or compare against `max(maxLtvBps_, currentMaxLtvBps)` so lowering the ceiling is always permitted. Additionally, expose an emergency/admin-gated path that can force `maxLtvBps` down without routing through `checkLtvBand`.

REMEDIATION COMMENT

Addressed in: [🔗 3d0a973](#)

SOLVED: The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.

HAL-005

SOLVED

migrateVenue is blocked by the same CF-cut gate, invalidating staged multisig migrations

● Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`migrateVenue` shares the same `checkLtvBand` rung-5 gate as `setMaxLtvImpl`. A CF cut landing in the band $(1e8 / \text{minHealthBps}, \text{CF_init})$ e.g. CF: 8800 → 8200 at `minHealthBps = 12000` passes rung 3 ($\text{maxLtvBps} = 7500 < 8200$) but fails rung 5 ($12000 \times 8200 = 98,400,000 \leq 1e8$). Pre-fix, this migration would have succeeded; post-fix it reverts unconditionally, silently invalidating a multisig venue hash staged ahead of execution.

The impact is limited to operational disruption rather than fund loss: a staged multisig migration fails silently at execution time with no advance warning, forcing the team to re-stage after adjusting `minHealthBps`. No user funds are directly at risk, but repeated CF volatility in this band can cause migrations to fail unpredictably, consuming multisig signer time and eroding confidence in the staging process itself.

RECOMMENDATION

If `migrateVenue` needs an independent gate, snapshot the CF check at staging time rather than re-validating against the live CF at execution time.

REMEDIATION COMMENT

Addressed in: [🔗 756b998](#)

SOLVED: The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.

Stale stagedVenueHash survives an Ownable2Step vault-owner rotation, letting the proposer fire a venue rewrite the new owner never authorised

• Low · 2.0

A0:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:C/Y:N

DESCRIPTION

`LeveragedAerodromeCLStrategy.stageVenue` gates on `msg.sender == _vaultOwner()`, which resolves via `Ownable(vault()).owner()`, a live read against the vault, not a cached copy, and is the venue-selection authority the docstring at lines 1282-1285 reserves explicitly to the vault owner "so the hot proposer key can never choose where the fund's liquidity goes." The staged hash itself is stored in a single storage slot in the venue's diamond storage (the appended `stagedVenueHash` field) with no owner-binding: `stageImpl` writes `$.stagedVenueHash = venueHash;` and emits `VenueStaged(venueHash)` unadorned. The only clearing paths in production are:

- `migrateImpl` - consumes the hash on migration.
- `redployImpl` - clears if set on redeploy.
- `_clearStagedVenue` - invoked from the three admin setters `setTargetLtvImpl`, `setMaxLtvImpl`, and `setWidthBoundsImpl`.

None of the above is wired to `Ownable2Step.transferOwnership` / `acceptOwnership` on the vault, and there is no on-chain callback from `_transferOwnership` into the strategy. So `stagedVenueHash` persists across a two-step vault ownership rotation, and the on-chain authority-check in `migrateImpl` at `(if (staged == bytes32(0) || keccak256(abi.encode(p)) != staged) revert VenueNotStaged();)` verifies only that `p` matches the staged hash, it does not verify that the current vault owner authored that hash.

Under normal operation the proposer's authority is bounded by "where the vault-owner has committed the fund's liquidity can go"; this bug widens that bound for the duration of the ownership-rotation window whenever a stale stage exists. The proposer chooses when to fire (or when to combine with a `flatten` that they control), so the timing of the exploitation is at the hot key's discretion. Every parameter reachable through `applyVenue` is inside blast radius: pool address, gauge, three markets, three feeds, target/max LTV, min health, width band.

RECOMMENDATION

Bind the staged hash to the staging owner so an ownership rotation invalidates any prior stage.

REMEDIATION COMMENT

Addressed in: [🔗 414f04d](#)

SOLVED: The **Moonwell** team solved the issue in the specified commit id by following the mentioned recommendation.

withdrawIdle fallback loosens the LTV bound when Moonwell prices USDC above \$1

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`LeveragedAeroVenue._unleveredAtVenueOracle` is the fallback bound `withdrawIdleImpl` uses when the strategy's hardened oracle read fails; on catch it calls this function and emits `WithdrawIdleBoundDegraded`). Its job is to derive the un-levered collateral ceiling from Moonwell's own view instead of Chainlink, so a withdrawal keeps a hard invariant even while the Chainlink hop is degraded. It does that by reading:

SOL 251-257

```

251  uint256 c = (mUSDC.balanceOf(this) * mUSDC.exchangeRateStored()) / 1e18; // 6dp USDC
      face
252  uint256 cf = readCollateralFactor(...); // bps
253  (err, liquidity, shortfall) = comptroller.getAccountLiquidity(this); // 18dp USD
254  uint256 dVenue = (c * cf) / 10_000 + shortfall / 1e12;
255  uint256 liqFace = liquidity / 1e12; // <-- USDC
      pinned at $1
256  dVenue = dVenue > liqFace ? dVenue - liqFace : 0;
257  return _unleveredFrom(c, dVenue);

```

`getAccountLiquidity` returns Moonwell's own 18-dp USD figure of $C \cdot CF - D$, where C and D are the same USD-values Moonwell prices with, i.e. $C = mUsdcCollateral \cdot P_{usdc_moonwell}$ and $D = D_face \cdot P_{usdc_moonwell}$ when USDC is the only debt-carrying market here (there is no cbBTC/WETH borrow on the withdraw-idle path; `withdrawIdle` is USDC-in, USDC-out, and the CL leg carries the two other markets through the CL position, not through the borrower). The library then converts both quantities to 6-dp USDC face by dividing by `1e12`, which is only exact under the pinning assumption $P_{usdc_moonwell} == \$1$. Algebraically, if Moonwell's USDC oracle reports $P_{usdc} = 1 + \epsilon$:

CODE

```

liqFace = (C·CF - D_face·(1+ε)) / 1e12 // 18dp → 6dp, but the (1+ε) never got divided
out
D_recovered = C·CF/1e4 + shortfall/1e12 - liqFace
              = D_face · (1 + ε) (algebra of the two branches)

```

So when $\epsilon > 0$ (Moonwell prices USDC above \$1, the historical direction for the peg during stress, when Chainlink is also more likely to be stale, i.e. exactly the branch this fallback exists to serve), $D_{\text{recovered}} = D_{\text{face}} \cdot (1+\epsilon) > D_{\text{face}}$. But `_unleveredFrom` expects a face-USDC debt: $\text{backing} = \text{ceil}(D \cdot 10_{000} / \text{targetLtvBps})$. Since the library **understates** the debt when $\epsilon > 0$ — the $(1+\epsilon)$ factor lives on both $C \cdot CF$ and D , but the code only strips it (implicitly, via `/1e12`) from D , not from the $C \cdot CF / 10_{000}$ term, the returned un-levered budget is larger than target LTV permits. The inline docstring at lines 244-247 explicitly claims "USDC taken at face, a depeg only tightens the bound," which is the wrong direction: below-peg indeed tightens (the returned budget shrinks), above-peg loosens.

The attacker here is any proposer with authority to call `withdrawIdle` (`onlyProposer`; the same hot key that already runs `deploy` / `rerange` / `compound`), during the degraded window when the strategy's own oracle either reverts on staleness or returns a value that fails the calm-gate, the exact window the fallback exists for. The proposer does not need to do anything malicious to trip the drift: `withdrawIdle(amount)` for amount sized to the fallback bound will silently pass while `amount` would have exceeded the true un-levered bound, and the post-op LTV drifts above `targetLtvBps` toward `maxLtvBps`. On the next `deployIdle` at the standing target the two errors compound (the newly-idle USDC is treated as unlevered when it is now part of a slightly-over-levered book). No permissionless deleverage fires because `minHealthBps` is checked against $\text{LTV} = 1e8 / \text{minHealthBps}$, not against `targetLtvBps`, so the drift sits in the `[target, 1e8/minHealth]` band untouched by any autonomous corrective action.

RECOMMENDATION

Divide the 18-dp Moonwell figures by the live Chainlink USDC price rather than by `1e12`, so both branches carry the same denominator and the peg factor cancels. This is the cleanest fix but re-introduces an oracle dependency the "fail-closed at venue oracle" path was designed to sidestep. Keep the fallback mostly independent of the primary hardened oracle by reading **only** `$.usdcFeed` (the peg feed), not the AERO/legA/legB feeds that the primary path fail-closes on. This preserves the intent of "degrade gracefully when the *position-valuation* feeds are down" while still correcting the peg-pinning bug, since the USDC feed is unrelated to why the primary oracle failed in the first place.

Alternatively if a code fix is deferred, at minimum correct the docstring, which currently makes the affirmatively wrong claim that "USDC taken at face, a depeg only tightens the bound." Replace it with an accurate statement of the actual behavior (above-peg loosens the bound, bounded by ϵ and the accrual window before someone notices) and flag `_unleveredAtVenueOracle` as a known-imprecise fallback, so operators monitoring the degraded-oracle window know to manually verify LTV after any `withdrawIdle` that fires through this path.

REMEDIATION COMMENT

Addressed in: [🔗 9d4cab4](#)

SOLVED: The **Moonwell** team solved the issue in the specified commit id by following the mentioned recommendation.

HAL-008

SOLVED

applyVenue silently rewrites minHealthBps on migration without a dedicated event

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`LeveragedAeroVenue.applyVenue` writes `$.minHealthBps = p.minHealthBps` unconditionally on every venue migration, but unlike the three sibling policy fields updated in the same block, it does not emit a paired `MinHealthUpdated`. The inline comment at lines 705-709 explicitly justifies why the neighbouring fields DO announce themselves ("a migration moves them in silence past a monitor keyed on the setters' events"), so this is the same class of gap the file has already identified for `targetLtvBps` (line 710), `maxLtvBps` (line 711), and `minWidth` / `maxWidth` (line 700-701), and treated. `minHealthBps` is not merely a knob, it gates the permissionless deleverage safety-valve: `LeveragedAeroValuation.checkLtvBand` (line 334) documents at line 330 that the permissionless-deleverage trigger sits at `LTV = 1e8 / minHealthBps`. So moving `minHealthBps` from e.g. `12000` (deleverage trigger at LTV ≈ 8333 bps) toward the enforced floor of `10500` (trigger at LTV ≈ 9524 bps) is a ≈ 12 percentage-point loosening of the emergency backstop.

`minHealthBps` also has no admin setter, `setTargetLtvImpl` (line 738), `setMaxLtvImpl` (line 749), and `setWidthBoundsImpl` (line 760) are the three admin setters, and none touch `minHealthBps`. Migration is therefore the only mutation path for the field. A monitor keyed on `MinHealthUpdated`, the natural on-chain signal, never sees the drift because no such event exists.

RECOMMENDATION

Declare and emit a paired event, guarded on inequality the same way the surrounding three are.

REMEDIATION COMMENT

Addressed in: [🔗 756b998](#)

SOLVED: The **Moonwell** team solved the issue in the specified commit id by following the mentioned recommendation.

_settleShortfall reads every leg's Chainlink feed, so a stale sibling-leg feed reverts emergencyRedeem (deadman), flatten, and migrateVenue on a zero-debt leg

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`LeveragedAeroManager._settleShortfall` is the fail-closed Chainlink-priced backstop that runs after Phase 1 (oracle-free cover from on-hand USDC) leaves any residual borrow. Its zero-debt short-circuit sits **before** the price read, but the price read itself is unconditional:

SOL 1065-1072

```
1065 function _settleShortfall() private {
1066     Layout storage $ = _layout();
1067     uint256 cbDebtRem = IMoonwellMarket($.mCbBTC).borrowBalanceCurrent(address(this));
1068     uint256 wethDebtRem = IMoonwellMarket($.mWeth).borrowBalanceCurrent(address(this));
1069     if (cbDebtRem == 0 && wethDebtRem == 0) return; //
line 1069
1070     (uint256 pBTC, uint256 pETH, uint256 pUsdc) = _readAllPrices(); //
line 1072
1071     // ... size cbUsdcNeed, wethUsdcNeed; cover legs conditionally at lines 1085-1086
1072 }
```

`_readAllPrices` reads **all three** hardened Chainlink feeds, cbBTC/USD, WETH/USD, USDC/USD through `_readUsd8`, which fails-closed via `ChainlinkReader.readUsd`. Any stale feed / down sequencer / grace period reverts the whole call. So on a book where one leg still owes and the other leg's borrow was fully repaid in Phase 1 (or was never borrowed), the surviving leg's cover is gated on the stale-feed liveness of the leg that carries no debt at all. The **line-1085 / 1086** `if (cbDebtRem > 0) / if (wethDebtRem > 0)` guards below correctly skip the zero-debt cover, so the code already knows which price it doesn't need, but it has already read the price by that point.

RECOMMENDATION

Read only the feeds the cover actually needs, gated by the per-leg residual debt.

REMEDIATION COMMENT

Addressed in: [cc0dd31](#)

SOLVED: The **Moonwell** team solved the issue in the specified commit id by following the mentioned recommendation.

Redemption recipient is snapshotted at request time, letting an outgoing account owner capture funds meant for the new owner

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

A redemption request snapshots its payout recipient at request time rather than resolving it dynamically to the account's current `owner()`. Previously, `fulfillRedeem` paid the account and `claimWithdrawnUsdc` swept to whoever held `owner()` at claim time, so a `transferOwnership` between request and claim correctly re-routed the payout to the new owner. This PR reverses that: the payout now goes to the recipient recorded at request time. If Alice (the old owner) has an open redemption request and then transfers account ownership to Bob, the backend's scheduled fulfillment still pays Alice, not Bob, even though Bob now owns the account. Bob's only defense, `cancelWithdraw`, only works if `r.settled == false`, he must win a race against the backend operator's own fulfillment schedule. Neither party can fix it after the fact: `emergencyRedeem` is gated to `r.owner` (the account, not Alice), and the account's `emergencyWithdraw` is `onlyOwner` (Bob).

The impact is a direct misdirection of user funds during account ownership transfers: USDC that should settle to the new owner is paid instead to the prior owner, with no on-chain remedy once `fulfillRedeem` executes.

RECOMMENDATION

Resolve the redemption payout recipient dynamically at settlement time (current `owner()`), matching the prior behavior. If the snapshot is intentional, `transferOwnership` should sweep or auto-cancel all open requests for the transferring account.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

Direct-pay to the recipient frozen at request time is the deliberate design from PR #85 (removes the unclaimed-proceeds state and the second claim transaction). Account ownership transfer is a registry-mediated, user-only call not exposed by the product, and the outgoing owner already had strictly easier paths to the same funds. Resolving `owner()` at settlement would add a requester-controlled revert hook into the proposer's `fulfillRedeem`.

Proposer can DoS an owner-staged cross-tickSpacing migration by poisoning \$.skewBps between stage and migrate

• Informational · 1.1 A0:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:L/D:N/Y:N

DESCRIPTION

`LeveragedAeroVenue.VenueParams` commits via the staged `keccak256(abi.encode(p))` to the destination pool, gauge, all three feeds, `tickSpacing`, both swap spacings, width, `minWidth`, `maxWidth`, `targetLtvBps`, `maxLtvBps`, `minHealthBps`. It does **not** commit to `skewBps`, `minSkewBps`, or `maxSkewBps`. Meanwhile `applyVenue` validates the destination range against storage-side skew fields:

SOL 671-674

```
671 LeveragedAeroValuation.checkBands(p.tickSpacing, p.minWidth, p.maxWidth, $.minSkewBps,
672   $.maxSkewBps);
673 LeveragedAeroValuation.checkRange(
674   p.width, $.skewBps, p.tickSpacing, p.minWidth, p.maxWidth, $.minSkewBps, $.maxSkewBps
675 );
```

`checkRange` computes `lowerSpan = width · skewBps / 10000`, `upperSpan = width - lowerSpan`, and reverts `OutOfBounds` when either span is smaller than one `tickSpacing`. So the same `skewBps` value that was legal at the current venue (checked against the current `tickSpacing`) can be illegal at the destination venue (checked against `p.tickSpacing`) when the two spacings differ.

The write path for `$.skewBps` outside init and outside `applyVenue` is

`LeveragedAerodromeCLStrategy.rerange`. Its body persists the caller's `skewBps_` before delegating to `rerangeImpl`:

SOL 503-508

```
503 LeveragedAeroValuation.checkRange(
504   width_, skewBps_, $.tickSpacing, $.minWidth, $.maxWidth, $.minSkewBps, $.maxSkewBps
505   // <-- validates against CURRENT tickSpacing
506 );
507 $.width = width_;
508 $.skewBps = skewBps_;
509 LeveragedAeroManager.rerangeImpl(minLiq0, minLiq1);
```

`rerangeImpl` at `LeveragedAeroManager.sol:503-505` early-returns on a flat book (`if ($.tokenId == 0) return;`), the docstring at strategy line 971-973 explicitly confirms "on a FLAT book it returns early and the persist is all that happens." So on the flat book that `migrateVenue` already requires as its precondition (`migrateImpl`'s `BookNotFlat` check at `LeveragedAeroVenue.sol:535`), the proposer can call `rerange(anyLegalWidth, extremeSkew, 0, 0)` and only the storage persist runs, the actual re-range logic is skipped.

RECOMMENDATION

Extend the staged hash to commit to the skew triple, and route `applyVenue`'s validation and persist through `p` rather than through storage.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

migrateVenue, flatten and redeploy are all onlyProposer, so the poisoner is the only party able to fire the migration it blocks; this is no stronger than the proposer declining to act, is visible on-chain, and is undone by the same key. The rejection is atomic and already pinned by `testMigrateRejectsADestinationThatStarvesTheLiveSkew`. The closed-form skew-proof destination width is documented in 86a3e7f.

! Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.